

ROS2K v6.2 — Student Cheat Page

Print this page. It contains every command, KPI, and scenario you need during the workshop. All commands run from `~/R2K-HSL/core`.

1. Launch flags (`launch_r2k.sh`)

Flag	Default	Explanation
<code>--scenario <name></code>	<code>2vs2_default</code>	Which match setup to load (bot positions, field). 17 scenarios available — see §4.
<code>--strategy <name></code>	<code>strat_aggro</code>	Which prompt fragments the blue LLM uses. <code>strat_aggro</code> = pressing, <code>strat_default</code> = balanced, <code>strat_recover</code> = defensive.
<code>--model <name></code>	<code>qwen2.5-coder:3b</code>	Ollama model. Must be pre-pulled (<code>ollama pull <name></code>). Larger = smarter but slower.
<code>--relay <name></code>	<code>only_sim_bots</code>	Hardware mapping. <code>only_sim_bots</code> = all Gazebo. <code>hardware_mirror</code> = real Yahboom + K1.
<code>--explain</code>	<code>off</code>	LLM outputs <code>analysis + oracle + assignments</code> (600 tokens). Good for debugging reasoning, but +44% latency.
<code>--no-explain</code>	<code>on</code>	LLM outputs ONLY <code>assignments</code> (150 tokens). Faster. Default for experiments.
<code>--headless</code>	<code>off</code>	Gazebo without GUI (<code>gzserver</code> only). ~30% faster. Use for batch experiments and automated runs.
<code>--duration <s></code>	<code>0 (manual)</code>	Auto-terminate after N seconds. <code>60</code> = quick test, <code>120</code> = B-study-compatible. <code>0</code> = run until CTRL+C or window close.

Typical workshop command:

```
./launch_r2k.sh --scenario 3vs3_attack_center --relay only_sim_bots
```

2. Testing (pytest)

7 test files, 91 tests total. 62 pass without ROS2 (unit tests); 29 require a live ROS2 + Gazebo stack (integration tests, auto-skipped if `rclpy` missing).

Test file	Tests	What it covers
<code>test_foul_detection.py</code>	7	Pushing (dist 0.3m, vel 0.5m/s), blocking (dist 0.5m, angle 30deg), hysteresis (N frames), ball proximity exemption, sideline Y range, foul cooldown
<code>test_integration_smoke.py</code>	7	End-to-end: scenario launches, momentum produces values, reward produces values, foul detection works, headless+duration, strategy files exist, launch script flags. Skips if no ROS2.
<code>test_kickoff_and_ballout.py</code>	28	Sideline warp (top/bottom), goal-line warp (blue/red), warp always inside field, kickoff scoring-team freeze (blue/red), ball reset to center, freeze time = 5s, no-toucher neutral restart, toucher penalty, pushing/blocking/ball-out penalty values, frozen bots get zero Twist, expired bots removed, goal detection (within posts, wide, edge, in-field)
<code>test_momentum.py</code>	8	OLS slope positive/negative, minimum samples (10), clamping ± 10 , flat scores, trend classification (ascending/collapsing)
<code>test_referee.py</code>	7	Pushing foul, blocking without ball, no foul with ball, sideline warp, ball-out sideline, goal-line out (no goal), last-touch tracking
<code>test_reward.py</code>	7	Positive reward, negative reward (foul), 1Hz update rate, scale clamping ± 10 , neutral classification, foul penalty schema, decision timeout
<code>test_set_piece.py</code>	27	Goal kick ball placement (4 corners), corner kick-in placement (4 corners), goal-line out classification (attacker vs defender, blue vs red), no-toucher neutral fallback, warp radius (bot within 1.5m warped 2m radially), warp direction, kickoff/goal_kick/corner countdown = 5s, scoring team frozen, status types distinct

How to run tests

```
# All tests (62 unit tests pass without ROS2, 29 integration tests skip)
python3 -m pytest tests/ -v

# Single test file
python3 -m pytest tests/test_foul_detection.py -v -s

# Only referee + set-piece tests
python3 -m pytest tests/test_referee.py tests/test_set_piece.py -v

# Verbose with print output (-s shows print statements)
python3 -m pytest tests/test_momentum.py -v -s
```

Tests are in `src/tests/` . Run from `src/` with `venv` active (U22) or inside the Docker container (U24). No config file needed — invoke explicitly.

3. KPIs (14 metrics from `analyze_trace.py`)

`analyze_trace.py` joins `llm_trace` + `world_trace` by `R2K_RUN_ID` and computes 14 KPIs. Run after a match:

```
python3 tools/analyze_trace.py --run-id <ID>
python3 tools/analyze_trace.py --run-id <ID> --plot          # +latency
                    histogram, score timeline
python3 tools/analyze_trace.py --run-id <ID> --output results/ # save JSON
                    instead of stdout
```

KPI	Source	Target	What it measures
goals_for_blue	world_trace	> goals_for_red	Blue goals scored (score delta count)
goals_for_red	world_trace	< goals_for_blue	Red goals scored
tactical_score_avg	world_trace	> -1.0	Mean average_numerical_score over the match
tactical_score_final	world_trace	> -2.0	Last current_numerical_score (end-of-match state)
cluster_pct	world_trace	< 10%	% frames where min pairwise blue distance <

KPI	Source	Target	What it measures
			1.5m (bots bunching up)
goalie_idle_pct	world_trace	< 70% (after Phase 2a)	% frames goalie moved < 0.1m (currently ~95% — structural limit)
oob_pct	world_trace	< 10%	% frames any blue bot > 0.5m outside field bounds (out-of-bounds)
ball_possession_blue_pct	world_trace	> 50%	% frames where closest bot to ball is blue
latency_p50	llm_trace	< 1000ms	50th percentile LLM response time (median)
latency_p95	llm_trace	< 2000ms	95th percentile (tail latency)
latency_max	llm_trace	—	Worst-case single LLM call (watch for cold-boot spikes)
parse_error_rate	llm_trace	< 5%	% LLM calls with <code>parse_code > 0</code> (malformed/truncated JSON)
role_diversity	llm_trace	> 2	Count of distinct role strings assigned (e.g. striker, goalie, supporter)
status_distribution	world_trace	—	Counter of <code>match_state.status</code> values (playing, ball_out, kickoff, etc.)
avg_response_tokens	llm_trace	< 100	Mean <code>len(raw_response) / 4</code> (approximate token count)

Composite score (weighted blend, single number for comparison):

```
composite = 0.4 × goal_diff_norm + 0.3 × tac_score_norm + 0.2 × possession + 0.1 × latency_factor
```

- `goal_diff_norm`: `(goals_blue - goals_red)` normalized to 0..1
- `tac_score_norm`: `tactical_score_avg` mapped from -10..+10 to 0..1
- `possession`: `ball_possession_blue_pct / 100`
- `latency_factor`: `max(0, 1 - latency_p50 / 2000)`

A composite of 0.5+ is a good run. Baseline (B-study A) was ~0.55.

4. Test scenarios (10 tactical scenarios)

Each scenario has a `scenario/{name}/` folder with `field_diagram.png`, `analysis.md` (oracle + expert), and `kpi_targets.json`.

Primary scenarios (3vs3)

#	Scenario	Oracle (what blue should do)	Key KPI targets	What to watch
1	3vs3_attack_center	Exploit central gap between red bots. Quick central passing.	composite 0.4–1.0 · OOB <10% · cluster <10% · possession 40–100%	Role switches as ball moves. Goalie stays at X=-4.0.
2	3vs3_attack_wing	Drive down sideline, cross to center. Stretch red defense.	composite 0.35–1.0 · OOB <15% · cluster <12%	Sideline OOB — STAY INSIDE rule critical on wing.
3	3vs3_contain_delay	Zone-defend, block passing lanes. Force red into mistakes.	composite 0.3–0.75 · possession 25–70% · goals 0–5	Defensive solidity. Low possession is expected.
4	3vs3_defensive_crisis	Emergency clear — kick ball away from own goal. Survive.	composite 0.25–0.8 · goalie_idle <60% · cluster <15%	Score volatility. Goalie should be very active.
5	3vs3_def_transition	Fall back, re-form defensive line, then counter-press.	composite 0.3–0.85 · cluster <15% · possession 30–80%	Role diversity spike (rapid role switching). Chaos.
6	3vs3_fast_counter	Exploit open space with speed. Push forward fast.	composite 0.4–1.0 · OOB <12% · possession 40–100%	Deep Move targets (X > 2.0). Minimal role switches.
7	3vs3_high_line	High defensive line, sweeper	composite 0.35–0.95 ·	Breakaways — if red gets

#	Scenario	Oracle (what blue should do)	Key KPI targets	What to watch
		goalie. Compress the game.	goalie_idle <50% · possession 40–100%	behind, goalie must intercept.
8	3vs3_long_shot	Exploit distance — shoot when red goalie is out of position.	composite 0.35–0.9 · goals 0–8	Kick assignments at $X > 0.5$, $ Y < 1.5$. Low accuracy.
9	3vs3_pressing_trap	Escape the press. Maintain spacing, don't cluster. Short safe passes.	composite 0.3–0.8 · cluster <15% · OOB <15%	cluster_pct spike — if two bots converge, the press wins.

Secondary scenario (2vs2)

#	Scenario	Oracle	Key KPI targets	What to watch
10	2vs2_default	One attacks, one defends. Role clarity critical — no third bot to cover.	composite 0.35–0.9 · latency <800ms · cluster <15%	High role diversity (striker switches frequently). Simpler decision space — good for prompt iteration.

Running a specific scenario

```
# Quick 60s test
./launch_r2k.sh --headless --duration 60 --scenario 3vs3_pressing_trap --
relay only_sim_bots

# Full match with visualizer
./launch_r2k.sh --scenario 3vs3_attack_center --relay only_sim_bots

# With reasoning output (for oracle/expert comparison)
./launch_r2k.sh --headless --duration 60 --scenario 3vs3_fast_counter --
relay only_sim_bots --explain
```

5. Quick-test recipes

What	Command
Warm up Ollama (avoid dead blue team)	<code>curl -s http://127.0.0.1:11434/api/generate -d '{"model": "qwen2.5-coder:3b", "prompt": "hi", "stream": false}' > /dev/null</code>
30s smoke test (headless)	<code>./launch_r2k.sh --headless --duration 30</code>
Full match with visualizer	<code>./launch_r2k.sh --scenario 3vs3_attack_center</code>
Inspect prompt without ROS	<code>python3 tools/dump_prompt.py --scenario 3vs3_attack_center -no-explain</code>
Analyze last run's KPIs	<code>python3 tools/analyze_trace.py --run-id <ID></code>
Analyze + plots	<code>python3 tools/analyze_trace.py --run-id <ID> --plot</code>
Baseline experiment (3×120s)	<code>./tools/run_experiment.sh A baseline 120 3vs3_attack_center strat_default --no-explain</code>
Run all unit tests	<code>python3 -m pytest tests/ -v</code>
Run single test file	<code>python3 -m pytest tests/test_set_piece.py -v -s</code>
Check GPU + Ollama	<code>nvidia-smi</code> (look for ~2-4GB VRAM)
Find your run ID	<code>ls logs/llm_trace_*.jsonl</code> (filename = run ID)

6. File locations (where things live)

What	Path	Notes
Launch script	<code>core/launch_r2k.sh</code>	Single entrypoint for all runs
Python nodes	<code>core/src/*.py</code>	referee, score, reward, aggregator, red, bridge, evaluator, visualizer
Prompt fragments	<code>core/src/strategy/fragments/*.txt</code>	Edit these — <code>setup_r2k.py</code> assembles them at boot
Scenarios (flat)	<code>core/src/scenario/*.json</code>	Bot positions, field setup

What	Path	Notes
Scenario packages	core/src/scenario/{name}/	scenario.json + field_diagram.png + analysis.md + kpi_targets.json
Relay profiles	core/src/relay/*.json	Hardware mapping
Trace logs	core/src/logs/*_*.jsonl	llm_trace (per LLM call) + world_trace (per 10Hz tick). Gitignored.
Tools	core/src/tools/*.py	analyze_trace , dump_prompt , gen_field_diagrams , run_experiment , swap_fragments
Tests	core/src/tests/test_*.py	7 files, 91 tests
Knowledge base	core/src/ros2k_knowledge/*.md	7 power-files + router. opencode reads these automatically.
Referee rulebook	core/docs/referee_rulebook.md	Complete decision catalog — read before changing any rule
Optimization spec	core/docs/optimization_spec_v6.2.md	Phases 0-5, experiment catalog, KPI definitions